



Batch 8 — Website Backend Lead Capture API

Daniel Macharia <danielmacharia43@gmail.com>
To: <daniel@ics.ac.ke>

Tue, 16 Jun at 08:11

Batch 8 — Website Backend Lead Capture API

Meat Lovers CIMS powered by YohPal

8A.

database/migrations/018_website_leads.sql

```
CREATE TABLE website_leads (  
  id BIGINT AUTO_INCREMENT PRIMARY KEY,  
  
  full_name VARCHAR(255) NOT NULL,  
  phone VARCHAR(50) NOT NULL,  
  email VARCHAR(255),  
  
  interest_type ENUM(  
    'GENERAL',  
    'AI_ORDER_ASSISTANT',  
    'FOOD_ORDER',  
    'SOFT_DRINKS',  
    'ALCOHOLIC_DRINKS',  
    'OFFERS',  
    'CATERING',  
    'DELIVERY',  
    'FEEDBACK'  
  ) DEFAULT 'GENERAL',  
  
  message TEXT,  
  
  lead_status ENUM(  
    'NEW',  
    'CONTACTED',  
    'CONVERTED',  
    'CLOSED'  
  ) DEFAULT 'NEW',  
  
  source VARCHAR(255) DEFAULT 'WEBSITE',  
  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP  
);
```

8B.

database/migrations/019_catering_enquiries.sql

```
CREATE TABLE catering_enquiries (  
  id BIGINT AUTO_INCREMENT PRIMARY KEY,  
  
  lead_id BIGINT,  
  
  full_name VARCHAR(255) NOT NULL,  
  phone VARCHAR(50) NOT NULL,  
  
  event_date DATE,  
  guest_count INT,  
  location TEXT,  
  message TEXT,  
  
  enquiry_status ENUM(  
    'NEW',  
    'CONTACTED',  
    'QUOTED',  
    'CONFIRMED',  
    'CLOSED'  
  ) DEFAULT 'NEW',  
  
  FOREIGN KEY (lead_id)
```

```
REFERENCES website_leads(id),

created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);
```

8C.

database/migrations/020_delivery_enquiries.sql

```
CREATE TABLE delivery_enquiries (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,

  lead_id BIGINT,

  full_name VARCHAR(255) NOT NULL,
  phone VARCHAR(50) NOT NULL,

  delivery_address TEXT NOT NULL,
  order_request TEXT NOT NULL,

  enquiry_status ENUM(
    'NEW',
    'CONTACTED',
    'CONFIRMED',
    'DISPATCHED',
    'CLOSED'
  ) DEFAULT 'NEW',

  FOREIGN KEY (lead_id)
    REFERENCES website_leads(id),

  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);
```

8D.

database/migrations/021_customer_feedback.sql

```
CREATE TABLE customer_feedback (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,

  lead_id BIGINT,

  full_name VARCHAR(255),
  phone VARCHAR(50),

  rating INT NOT NULL,
  message TEXT,

  feedback_status ENUM(
    'NEW',
    'REVIEWED',
    'ACTION TAKEN',
    'CLOSED'
  ) DEFAULT 'NEW',

  FOREIGN KEY (lead_id)
    REFERENCES website_leads(id),

  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);
```

8E.

backend/app/Controllers/WebsiteController.php

```
<?php

declare(strict_types=1);

namespace Controllers;

use Support\Database;
use Support\Request;
use Support\Response;
use Support\AuditLogger;
```

```

class WebsiteController
{
    public function storeLead(): void
    {
        $data = Request::body();
        $db = Database::connection();

        $db->beginTransaction();

        $leadId = $this->createLead($data, $data['interest_type'] ?? 'GENERAL');

        $this->createOrUpdateCustomer(
            $data['full_name'],
            $data['phone'],
            $data['email'] ?? null,
            $data['message'] ?? null,
            'WEBSITE'
        );

        AuditLogger::log(
            null,
            'WEBSITE',
            'CREATE_WEBSITE_LEAD',
            $leadId,
            null,
            $data
        );

        $db->commit();

        Response::success([
            'lead_id' => $leadId,
            ], 'Website lead submitted');
    }

    public function storeCateringEnquiry(): void
    {
        $data = Request::body();
        $db = Database::connection();

        $db->beginTransaction();

        $leadId = $this->createLead([
            'full_name' => $data['full_name'],
            'phone' => $data['phone'],
            'email' => $data['email'] ?? null,
            'message' => $data['message'] ?? null,
            ], 'CATERING');

        $stmt = $db->prepare(
            'INSERT INTO catering_enquiries
            (lead_id, full_name, phone, event_date, guest_count, location, message)
            VALUES (:lead_id, :full_name, :phone, :event_date, :guest_count, :location, :message)'
        );

        $stmt->execute([
            'lead_id' => $leadId,
            'full_name' => $data['full_name'],
            'phone' => $data['phone'],
            'event_date' => $data['event_date'] ?? null,
            'guest_count' => $data['guest_count'] ?? null,
            'location' => $data['location'] ?? null,
            'message' => $data['message'] ?? null,
        ]);

        $this->createOrUpdateCustomer(
            $data['full_name'],
            $data['phone'],
            $data['email'] ?? null,
            'Catering enquiry: ' . ($data['message'] ?? ''),
            'WEBSITE_CATERING'
        );

        AuditLogger::log(
            null,
            'WEBSITE',
            'CREATE_CATERING_ENQUIRY',
            $leadId,
            null,
            $data
        );

        $db->commit();

        Response::success([
            'lead_id' => $leadId,
            ], 'Catering enquiry submitted');
    }
}

```

```

}

public function storeDeliveryEnquiry(): void
{
    $data = Request::body();
    $db = Database::connection();

    $db->beginTransaction();

    $leadId = $this->createLead([
        'full_name' => $data['full_name'],
        'phone' => $data['phone'],
        'email' => $data['email'] ?? null,
        'message' => $data['order_request'] ?? null,
    ], 'DELIVERY');

    $stmt = $db->prepare(
        'INSERT INTO delivery_enquiries
        (lead_id, full_name, phone, delivery_address, order_request)
        VALUES (:lead_id, :full_name, :phone, :delivery_address, :order_request)'
    );

    $stmt->execute([
        'lead_id' => $leadId,
        'full_name' => $data['full_name'],
        'phone' => $data['phone'],
        'delivery_address' => $data['delivery_address'],
        'order_request' => $data['order_request'],
    ]);

    $this->createOrUpdateCustomer(
        $data['full_name'],
        $data['phone'],
        $data['email'] ?? null,
        'Delivery enquiry: ' . ($data['order_request'] ?? ''),
        'WEBSITE_DELIVERY'
    );

    AuditLogger::log(
        null,
        'WEBSITE',
        'CREATE_DELIVERY_ENQUIRY',
        $leadId,
        null,
        $data
    );

    $db->commit();

    Response::success([
        'lead_id' => $leadId,
    ], 'Delivery enquiry submitted');
}

public function storeFeedback(): void
{
    $data = Request::body();
    $db = Database::connection();

    $db->beginTransaction();

    $leadId = $this->createLead([
        'full_name' => $data['full_name'] ?? 'Anonymous Customer',
        'phone' => $data['phone'] ?? 'UNKNOWN',
        'email' => null,
        'message' => $data['message'] ?? null,
    ], 'FEEDBACK');

    $stmt = $db->prepare(
        'INSERT INTO customer_feedback
        (lead_id, full_name, phone, rating, message)
        VALUES (:lead_id, :full_name, :phone, :rating, :message)'
    );

    $stmt->execute([
        'lead_id' => $leadId,
        'full_name' => $data['full_name'] ?? null,
        'phone' => $data['phone'] ?? null,
        'rating' => $data['rating'],
        'message' => $data['message'] ?? null,
    ]);

    if (!empty($data['full_name']) && !empty($data['phone'])) {
        $this->createOrUpdateCustomer(
            $data['full_name'],
            $data['phone'],
            null,
        );
    }
}

```

```

        'Feedback: ' . ($data['message'] ?? ''),
        'WEBSITE_FEEDBACK'
    );
}

AuditLogger::log(
    null,
    'WEBSITE',
    'CREATE_CUSTOMER_FEEDBACK',
    $leadId,
    null,
    $data
);

$db->commit();

Response::success([
    'lead_id' => $leadId,
], 'Feedback submitted');
}

private function createLead(array $data, string $interestType): int
{
    $stmt = Database::connection()->prepare(
        'INSERT INTO website_leads
        (full_name, phone, email, interest_type, message, source)
        VALUES (:full_name, :phone, :email, :interest_type, :message, :source)'
    );

    $stmt->execute([
        'full_name' => $data['full_name'],
        'phone' => $data['phone'],
        'email' => $data['email'] ?? null,
        'interest_type' => $interestType,
        'message' => $data['message'] ?? null,
        'source' => 'WEBSITE',
    ]);

    return (int) Database::connection()->lastInsertId();
}

private function createOrUpdateCustomer(
    string $fullName,
    string $phone,
    ?string $email,
    ?string $notes,
    string $source
): void {
    $db = Database::connection();

    $existing = $db->prepare(
        'SELECT id FROM customers WHERE phone = :phone LIMIT 1'
    );

    $existing->execute(['phone' => $phone]);
    $customer = $existing->fetch();

    if ($customer) {
        $stmt = $db->prepare(
            'UPDATE customers
            SET full_name = :full_name,
            email = COALESCE(:email, email),
            notes = CONCAT(COALESCE(notes, ""), "\n", :notes),
            customer_type = "REGULAR"
            WHERE id = :id'
        );

        $stmt->execute([
            'full_name' => $fullName,
            'email' => $email,
            'notes' => '[' . $source . ']' . ($notes ?? ''),
            'id' => $customer['id'],
        ]);
    }

    return;
}

$stmt = $db->prepare(
    'INSERT INTO customers
    (full_name, phone, email, customer_type, loyalty_points, notes)
    VALUES (:full_name, :phone, :email, :customer_type, 0, :notes)'
);

$stmt->execute([
    'full_name' => $fullName,
    'phone' => $phone,
    'email' => $email,

```

```

        'customer_type' => 'REGULAR',
        'notes' => '[' . $source . ']' . ($notes ?? ''),
    });
}
}

```

8F. Update

backend/routes/api.php

Add import:

```
use Controllers\WebsiteController;
```

Add routes:

```

$router->post('/api/website/leads', [WebsiteController::class, 'storeLead']);
$router->post('/api/website/catering-enquiries', [WebsiteController::class, 'storeCateringEnquiry']);
$router->post('/api/website/delivery-enquiries', [WebsiteController::class, 'storeDeliveryEnquiry']);
$router->post('/api/website/feedback', [WebsiteController::class, 'storeFeedback']);

```

8G.

docs/WEBSITE_LEAD_CAPTURE_API.md

Website Lead Capture API

System Name

Meat Lovers CIMS powered by YohPal

Purpose

The website is not just an online brochure. It is an AI-enabled customer acquisition engine.

This API captures:

- general leads
- AI assistant enquiries
- food order interest
- soft drinks interest
- alcoholic drinks interest
- offers interest
- catering enquiries
- delivery enquiries
- customer feedback

Tables Added

1. `website\_leads`
2. `catering\_enquiries`
3. `delivery\_enquiries`
4. `customer\_feedback`

CRM Integration

Every website submission creates or updates a customer record in `customers`.

If the phone number already exists:

- customer is updated
- notes are appended
- customer type becomes `REGULAR`

If the phone number does not exist:

- a new customer is created

Audit Logging

Each website submission creates an audit log entry.

Audit module:

```

```text
WEBSITE

```

Actions:

```

CREATE_WEBSITE_LEAD
CREATE_CATERING_ENQUIRY
CREATE_DELIVERY_ENQUIRY
CREATE_CUSTOMER_FEEDBACK

```

## API Endpoints

```
POST /api/website/leads
POST /api/website/catering-enquiries
POST /api/website/delivery-enquiries
POST /api/website/feedback
```

## Business Impact

This turns the Meat Lovers website into a measurable acquisition engine.

Management can track:

- customer interest
- catering demand
- delivery demand
- feedback trends
- lead sources
- repeat customer activity

---

# Batch 8 Outcome

Website backend now supports:

- ✓ website leads table
- ✓ catering enquiries table
- ✓ delivery enquiries table
- ✓ feedback table
- ✓ website controller
- ✓ website routes
- ✓ CRM integration
- ✓ audit logging for website submissions
- ✓ Meat Lovers CIMS powered by YohPal acquisition engine

# Next Smart Move

Build **Batch 9 – Admin Dashboard Website Leads Module**, including:

- leads listing page
- catering enquiries page
- delivery enquiries page
- feedback page
- lead status update API
- admin routes
- frontend API integration
- dashboard cards for acquisition performance